

ANN for Regression Using Keras

MACHINE LEARNING VS DEEP LEARNING

A Practical Study Guide for Data Science & AI Beginners

Predicting Tips using the Seaborn Dataset

Focus: Practical Implementation & Code Explanation

SECTION 1: INTRODUCTION

What is Regression?

In Machine Learning, **Regression** is a type of predictive modeling technique used when the target variable (the thing we want to predict) is continuous. Continuous means it can be any numerical value,

such as 15.5, 1000, or -3.2. This is different from Classification, where we predict categories (like "Cat" or "Dog"). If you are predicting a number that can fall anywhere on a number line, you are solving a regression problem.

Why Regression Problems Exist in Real Life?

We use regression every day to plan for the future. Real-world businesses and scientific fields rely heavily on numeric predictions to make decisions. Common real-life examples include:

- **Real Estate:** Predicting the exact price of a house based on its square footage, number of bedrooms, and location.
- **Finance:** Forecasting the stock price of a company for the next week.
- **Meteorology:** Predicting tomorrow's exact temperature in degrees.
- **Hospitality (Our Project):** Predicting how much tip a waiter will receive based on the total bill and the number of people at the table.

Why Neural Networks Can Be Used for Regression?

When most beginners hear "Neural Networks" or "Deep Learning," they think of image recognition or chatbots. However, Artificial Neural Networks (ANNs) are incredibly powerful for regression too. While simple algorithms like Linear Regression draw a straight line through data, Neural Networks can bend and curve to map highly complex, non-linear relationships. By simply changing the final layer of a Neural Network to output a continuous number instead of a probability, we convert it into a powerful regression engine.

SECTION 2: DATASET

What is the Tips Dataset?

The **Tips dataset** is a very popular, built-in dataset in the Python library called Seaborn. It contains records of tips given by customers at a restaurant. The waiter recorded information about each meal for a few months to see if there were patterns in how much people tipped.

Explanation of Columns

If you load the dataset, you will see several columns:

- **total_bill** : The total cost of the meal in dollars (Numeric).
- **tip** : The tip amount left by the customer in dollars (Numeric).
- **sex** : Gender of the bill payer (Male/Female).
- **smoker** : Whether there was a smoker in the party (Yes/No).
- **day** : Day of the week (Thur, Fri, Sat, Sun).
- **time** : Time of day (Lunch, Dinner).
- **size** : The number of people dining at the table (Numeric).

Sample Dataset Output

total_bill	tip	sex	smoker	day	time	size
16.99	1.01	Female	No	Sun	Dinner	2
10.34	1.66	Male	No	Sun	Dinner	3
21.01	3.50	Male	No	Sun	Dinner	3

Why Choose total_bill and size as Input Features?

Input features are the pieces of information we use to make our prediction. We chose **total_bill** and **size** because they are mathematically numerical. In real life, it makes logical sense: the more money someone spends (total bill), and the more people eating (size), the higher the tip usually is. Using these numerical columns avoids the complexity of encoding categorical text data (like days of the week) for beginners.

Why Choose tip as the Target Variable?

The target variable is what we want the AI to predict. We chose **tip** because estimating the tip is a classic continuous number prediction problem. The tip can be \$1.50, \$5.00, or \$20.00. This perfectly aligns with our goal of learning Regression.

SECTION 3: IMPORT LIBRARIES

Before writing the core logic, we must import all the tools we need. Let's look at the complete import block and break it down.

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Data Handling & Visualization

- `import pandas as pd`

What it does: Pandas is the primary library for data manipulation and analysis.

Why it is needed: It provides the "DataFrame"—a 2D table-like structure (rows and columns) that allows us to filter, slice, and read data easily. We use it to hold our dataset.

Without it: We would have to write complex, messy Python code using basic lists or dictionaries just to view a column.

- `import seaborn as sns`

What it does: Seaborn is a data visualization library based on matplotlib, but it also comes pre-loaded with practice datasets.

Why it is needed: We are using it specifically to call `sns.load_dataset('tips')`, which automatically downloads our restaurant data into a Pandas DataFrame.

Without it: We would need to manually download a CSV file of the tips dataset from the internet and load it from our local hard drive.

Machine Learning (Scikit-Learn)

- `from sklearn.model_selection import train_test_split`

What it does: A function that randomly shuffles and splits our data into two chunks (training set and testing set).

Why it is needed: We need to train our model on some data, and test it on data it has never seen before.

Without it: We would have to manually write a loop to select 80% of rows for training and 20% for testing, risking sorting bias.

- `from sklearn.preprocessing import StandardScaler`

What it does: It scales numerical data so that everything has a mean of 0 and a standard deviation of 1.

Why it is needed: Neural Networks perform terribly if one input feature is vastly larger than another (e.g., `total_bill` is in 10s or 100s, but `size` is 1 to 6). Scaling levels the playing field.

Without it: The neural network might prioritize `total_bill` over `size` simply because the numbers are bigger, ruining the learning process.

- `from sklearn.linear_model import LinearRegression`

What it does: It imports the classic statistical Machine Learning algorithm for linear regression.

Why it is needed: We want to build a baseline Machine Learning model to compare against our Deep Learning model.

Without it: We could only build the neural network and wouldn't know if Deep Learning actually provided any benefit over simple math.

- `from sklearn.metrics import r2_score, mean_squared_error`

What it does: These are mathematical scoring functions to check how accurate our predictions are.

Why it is needed: To put a number on our model's performance. R2 tells us accuracy percentage-wise, MSE tells us the average error amount.

Without it: We would just be guessing if our model is good or bad by visually looking at the predictions.

Deep Learning (Keras/TensorFlow)

- `from tensorflow.keras.models import Sequential`

What it does: It provides a blank canvas (a container) to build a Neural Network layer by layer, sequentially.

Why it is needed: It initializes the brain structure of our Artificial Neural Network.

Without it: We cannot define the architecture of our Deep Learning model.

- `from tensorflow.keras.layers import Dense`

What it does: It imports the standard "Dense" (fully connected) artificial neuron layer.

Why it is needed: We use this to actually add neurons into our `Sequential` model. "Dense" means every neuron in this layer connects to every neuron in the previous layer.

Without it: Our neural network would have no brain cells (neurons) to process the data.

SECTION 4: FEATURE SELECTION

Before we can teach a machine, we must clearly separate the questions (features) from the answers (targets).

```
# Load the dataset
df = sns.load_dataset('tips')

# Separate Inputs (X) and Output (y)
X = df[['total_bill', 'size']]
y = df['tip']
```

Explain Line by Line

- `X = df[['total_bill', 'size']]`: We create a new variable `X`. Notice the double brackets `[[ ]]`. This creates a 2D DataFrame. `X` contains only the two columns we want our model to look at to make its decision. These are the **Inputs** or **Features**.
- `y = df['tip']`: We create a variable `y`. Notice the single brackets `[ ]`, which creates a 1D Series. This column contains the actual tips given. This is the **Output** or **Target**.

Why are Inputs and Outputs Separated?

Think of it like a student taking a math test. `X` is the test paper containing the problems. `y` is the teacher's answer key. You cannot give the student the answer key while they are studying the problems, or they will just memorize the answers instead of learning the logic. We separate them so the algorithm can look at `X`, make a guess, and then we (the programmers) check that guess against `y`.

SECTION 5: TRAIN TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Explanation of Parameters

- `test_size=0.2` : This tells the function to take 20% of the dataset and hide it away as the testing set. The remaining 80% will be used for training.
- `random_state=42` : This acts as a seed for the random number generator. Shuffling the data is random, but by setting a seed (like 42), we ensure that every time we run the code, it shuffles the exact same way. This makes our results reproducible.

Why Training and Testing Data Are Needed?

Training Data (X_train, y_train): This is the material the model studies. The model looks at `X_train` and its corresponding `y_train` to discover patterns (e.g., "Oh, when total_bill is high, tip is usually high").

Testing Data (X_test, y_test): This is the final exam. After the model is trained, we give it `X_test` and ask it to predict the tips. We then compare its predictions to `y_test` to see how smart it really is.

What Data Leakage Means

Data Leakage happens when information from outside the training dataset (like the test set) is accidentally shared with the model while it is training. If you accidentally train the model on your test data, the model will get a 100% perfect score on the test because it has already seen the answers! This is cheating, and it leads to models that fail miserably in the real world. Splitting the data strictly prevents data leakage.

SECTION 6: FEATURE SCALING

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Explain Line by Line

- `scaler = StandardScaler()` : We create an instance (a tool) of the standard scaler.

- `scaler.fit_transform(X_train)` : The tool first **fits** (calculates the mean and standard deviation of the training data), and then **transforms** (applies the math formula to) the training data to scale it.
- `scaler.transform(X_test)` : We ONLY transform the test data. We do not use `fit` here. We use the mean and variance learned from the training data to scale the test data. This prevents data leakage!

Why Scaling is Important & What Happens Without It

Consider our two inputs:

- `total_bill` averages around \$20.00.
- `size` averages around 2 people.

Without scaling, an Artificial Neural Network looks at these numbers and thinks, "Wow, `total_bill` is 10 times larger than `size`! Total bill must be 10 times more important!" The model gets biased towards larger numbers. Scaling forces both columns to have a mean of 0 and a variance of 1. Now, `total_bill` might be `0.45` and `size` might be `0.60`. The neural network can now compare them fairly based on their actual pattern, not their raw magnitude.

SECTION 7: MACHINE LEARNING MODEL

Let's first build a traditional Machine Learning model using Simple Linear Regression before moving to Deep Learning.

```
# Initialize the model
ml_model = LinearRegression()

# Train the model on the training data
ml_model.fit(X_train_scaled, y_train)

# Generate predictions on the test data
ml_predictions = ml_model.predict(X_test_scaled)
```

Explain the Code

- `fit()` : This is where the actual "learning" happens. The algorithm looks at `X_train_scaled` and `y_train` and calculates the optimal weights (coefficients) to draw the best possible line through the data points.

- `predict()` : Once trained, the model takes the unseen `X_test_scaled` and applies the learned math formula to output predicted tip values.

Mathematical Explanation

$$y = b_0 + b_1x_1 + b_2x_2$$

Where:

- y = The predicted Tip (Target)
- b_0 = The Y-intercept (Bias). The base tip if the bill and size were zero.
- b_1 = The Weight (importance) assigned to feature 1 (total_bill).
- x_1 = The actual value of total_bill.
- b_2 = The Weight assigned to feature 2 (size).
- x_2 = The actual value of size.

Architecture Diagram

```

[Input: total_bill (x1)] -----(Weight b1)-----\
                                                    +----> Linear Regression
Math ----> [Prediction (y)]
[Input: size (x2)] -----(Weight b2)-----/

```

How It Learns and Predicts

Linear Regression learns by trying to minimize the distance between its predicted line and the actual data points. It tweaks b_1 and b_2 until the total error across all training examples is as small as mathematically possible. To generate a prediction, it simply plugs the new customer's `total_bill` and `size` into the $y = b_0 + b_1x_1 + b_2x_2$ equation.

SECTION 8: MODEL EVALUATION

We need to know how well our Linear Regression model performed.

```
# Calculate R2 Score
ml_r2 = r2_score(y_test, ml_predictions)

# Calculate Mean Squared Error
ml_mse = mean_squared_error(y_test, ml_predictions)

print(f"Linear Regression R2 Score: {ml_r2}")
print(f"Linear Regression MSE: {ml_mse}")
```

What is R² Score?

The R-squared (R²) score represents the percentage of the variance in the dependent variable (tip) that is explained by the independent variables (bill and size). It ranges from 0 to 1 (or 0% to 100%).

Interpretation: An R² of 0.45 means that 45% of the reason a tip changes is explained by the total bill and party size. The remaining 55% is due to other factors (like waiter friendliness or food quality) that our model cannot see. Closer to 1.0 is better.

What is MSE?

Mean Squared Error (MSE) measures the average squared difference between the estimated values and the actual value.

Interpretation: If a model predicts a tip of \$3, but the actual tip was \$5, the error is 2. The square of the error is 4. MSE averages this squared error across all predictions. We square it to ensure negative errors don't cancel out positive errors, and to heavily penalize massive mistakes. Closer to 0 is better.

SECTION 9: ANN WITH SINGLE NEURON

Now, let's move to Deep Learning. The most basic Neural Network is just a single artificial neuron. Interestingly, a single neuron with a linear activation mathematically behaves exactly like the Linear Regression model we just built!

```
# Build the model architecture
single_neuron_model = Sequential([
    Dense(1, input_dim=2)
])
```

Explain the Architecture

- **Sequential** : This creates an empty box. It tells Keras we are going to stack layers one after the other in a sequence.
- **Dense(1)** : We add a fully connected layer with exactly **1 neuron**. This single neuron will spit out one continuous number (the tip prediction).
- **input_dim=2** : We tell the neuron to expect exactly 2 inputs (our two columns: **total_bill** and **size**).
- **Neuron**: The computational unit that receives inputs, multiplies them by weights, adds a bias, and passes the sum forward.
- **Weight**: The strength of the connection. Exactly equivalent to b_1 and b_2 in our linear regression equation.
- **Bias**: An extra trainable constant added to the sum, allowing the neuron to shift its output up or down. Equivalent to the intercept b_0 .

Architecture Visually

```
total_bill (Input 1) -----(weight 1)-----\
                                                +---> ( Neuron + Bias ) --->
                        Output (Predicted Tip)
size (Input 2) -----(weight 2)-----/
```

Why this ANN Behaves Similar to Linear Regression

A single Dense neuron without an activation function performs a simple dot product: (Input 1 × Weight 1) + (Input 2 × Weight 2) + Bias. If you look closely, this is the exact same formula as $y = b_1x_1 + b_2x_2 + b_0$. Therefore, a 1-neuron neural network is effectively a machine learning linear regression engine running on deep learning hardware.

SECTION 10: COMPILE MODEL

Before a brain can learn, you must tell it *how* to learn and *what* mistakes to look out for.

```
single_neuron_model.compile(  
    optimizer='adam',  
    loss='mse'  
)
```

Explanation

- `compile()` : This function configures the model for training. It sets up the internal engine.
- `optimizer='adam'` : The optimizer is the algorithm that adjusts the weights of the network to minimize mistakes. **Adam** (Adaptive Moment Estimation) is the most popular, efficient, and beginner-friendly optimizer in Deep Learning. It decides how large of a "step" to take when adjusting the weights.
- `loss='mse'` : The loss function is the teacher grading the exam. We use **MSE** (Mean Squared Error). For regression problems, MSE is the standard. It calculates how far off the prediction was from the actual tip, squares the difference, and tells the optimizer "This is how bad you did, fix it."

Why are these needed before training? The neural network is initialized with random weights. If it doesn't have a loss function to calculate its error, and an optimizer to fix that error, it will just spit out random numbers forever and never improve.

SECTION 11: TRAIN MODEL

```
history = single_neuron_model.fit(  
    X_train_scaled,  
    y_train,  
    epochs=100,  
    verbose=1  
)
```

Explanation

- `epochs=100` : An epoch is one complete pass through the entire training dataset. Setting it to 100 means the model will look at the entire dataset 100 times to continuously adjust its weights and learn the patterns.
- `verbose=1` : This simply tells Keras to print a progress bar to the screen so we can watch the loss decrease epoch by epoch.

The Training Process: Step-by-Step

When you run `fit()`, a loop happens for every epoch:

1. **Forward Propagation:** The inputs (bill and size) are pushed through the network. The neuron multiplies them by its current (initially random) weights, adds the bias, and makes a guess (e.g., predicting a \$1.00 tip).
2. **Loss Calculation:** The model checks the true answer (e.g., the actual tip was \$5.00). The loss function (MSE) calculates the error: $(1.00 - 5.00)^2 = 16.0$. That is a high loss!
3. **Backpropagation:** The model works backward from the output to the input, calculating the gradient (slope) of the error with respect to each weight. It figures out which weight is to blame for the mistake.
4. **Weight Update:** The Adam Optimizer steps in. It takes the information from Backpropagation and tweaks the weights slightly. Maybe it increases weight 1 and decreases weight 2 so that next time, the prediction is a bit closer to \$5.00.

This cycle repeats hundreds or thousands of times until the error becomes as small as possible.

SECTION 12: PREDICTION

```
# Predict using the single neuron model
dl_single_predictions = single_neuron_model.predict(X_test_scaled)
```

What Prediction Means and How It Works

Prediction is the phase where learning is turned off. The weights and biases are "frozen." We pass in our `X_test_scaled` data. The neural network performs exactly one **Forward Propagation** pass using the

finely-tuned weights it learned during training. It spits out the final calculated number, which is its best guess for the tip. There is no loss calculation or backpropagation during this step.

SECTION 13: MULTI-LAYER ANN

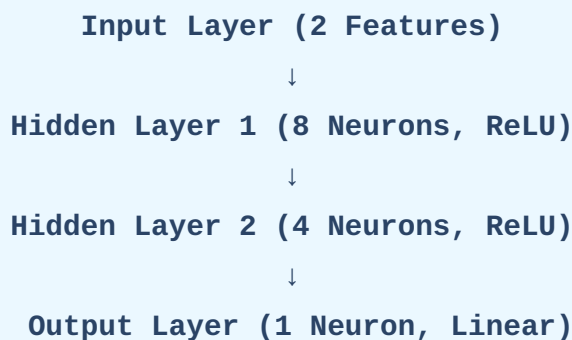
A single neuron is just linear regression. To unlock the true power of Deep Learning, we add **Hidden Layers** to create a Deep Neural Network capable of learning complex, non-linear patterns.

```
multi_layer_model = Sequential([
    Dense(8, activation='relu', input_dim=2),
    Dense(4, activation='relu'),
    Dense(1)
])

multi_layer_model.compile(optimizer='adam', loss='mse')

multi_layer_model.fit(X_train_scaled, y_train, epochs=100, verbose=0)
dl_multi_predictions = multi_layer_model.predict(X_test_scaled)
```

Architecture Breakdown



Explain Every Layer Individually

- `Dense(8, activation='relu', input_dim=2)`: This is the first **Hidden Layer**. It has 8 neurons. It receives the 2 inputs. We use **ReLU** (Rectified Linear Unit) as the activation function. ReLU turns any negative number into 0, but lets positive numbers pass through unchanged. This simple mathematical trick introduces *non-linearity*, allowing the network to learn curves instead of just straight lines.

- `Dense(4, activation='relu')` : The second hidden layer compresses the 8 outputs from the previous layer into 4 features, further refining the patterns learned.
- `Dense(1)` : The final output layer. It takes the processed signals from the 4 neurons and combines them into 1 final continuous number (the tip). It has no activation function (linear) because we want a raw regression number.

Why Multiple Layers? While a single layer learns simple associations (e.g., higher bill = higher tip), hidden layers allow the network to learn interactions between features (e.g., if the bill is high AND the party size is large, maybe the tip percentage drops slightly because they assume a mandatory gratuity is included). Deep networks learn abstractions.

SECTION 14: COMPARISON

Let's compare the three approaches we've discussed:

Feature	Linear Regression (ML)	Single Neuron ANN (DL)	Multi-Layer ANN (DL)
Architecture	Mathematical Equation	1 Neuron Output Layer	Input → Hidden Layers → Output
Hidden Layers	None	None	Yes (e.g., 8 neurons, then 4)
Complexity	Very Low	Low (Computationally simple)	High (Many weights and biases)
Learning Capability	Linear relationships only	Linear relationships only	Complex, Non-linear patterns
Best Use Case	Simple baseline, rapid deployment	Learning Keras basics	Massive datasets, complex features

SECTION 15: COMPLETE PROJECT CODE

Here is the full code from start to finish. Copy this into a Jupyter Notebook or Python file to see the results yourself!


```

# 1. Import all required libraries
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 2. Load the dataset
df = sns.load_dataset('tips')

# 3. Separate features (X) and target (y)
X = df[['total_bill', 'size']]
y = df['tip']

# 4. Split data to avoid data leakage (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Scale features so network treats both columns equally
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # Transform only!

# --- APPROACH 1: MACHINE LEARNING (LINEAR REGRESSION) ---
ml_model = LinearRegression()
ml_model.fit(X_train_scaled, y_train)
ml_preds = ml_model.predict(X_test_scaled)
print("ML R2 Score:", r2_score(y_test, ml_preds))

# --- APPROACH 2: DEEP LEARNING (MULTI-LAYER ANN) ---
# Build the architecture with Hidden Layers
dl_model = Sequential([
    Dense(8, activation='relu', input_dim=2), # Hidden Layer 1
    Dense(4, activation='relu'),              # Hidden Layer 2
    Dense(1)                                  # Output Layer for Regression
])

# Compile the model specifying optimizer and loss metric
dl_model.compile(optimizer='adam', loss='mse')

# Train the model (epochs=100)
dl_model.fit(X_train_scaled, y_train, epochs=100, verbose=0)

# Predict using the Deep Learning model
dl_preds = dl_model.predict(X_test_scaled)
print("DL Multi-Layer R2 Score:", r2_score(y_test, dl_preds))

```

Common Beginner Mistakes

- **Forgetting to Scale Data:** Without `StandardScaler`, the neural network's loss might explode to NaN (Not a Number) or fail to converge because `total_bill` numbers overwhelm `size` numbers.
- **Using an Activation in Output Layer:** If you accidentally put `Dense(1, activation='sigmoid')` or `'relu'` on the last layer, your model will break. Sigmoid forces predictions between 0 and 1, and ReLU prevents negative predictions. For basic regression, leave the final layer linear!
- **Data Leakage during Scaling:** Beginners often run `fit_transform()` on the whole dataset `X` before splitting. This allows information about the testing data to leak into the training process. Always split first, then fit scaler on training data only!

SECTION 16: CONCLUSION

What We Learned

In this guide, we demystified Regression using both traditional Machine Learning and Deep Learning with Keras. We learned how to process data, split it safely to avoid leakage, and scale numerical features. We proved that a single-neuron Neural Network is essentially doing the exact same mathematical job as a Scikit-Learn Linear Regression model. We then elevated our architecture by adding dense hidden layers with ReLU activations, transitioning into true Deep Learning.

Machine Learning vs Deep Learning in this Project

For a tiny dataset like *Tips* with only two input features, traditional Machine Learning (Linear Regression) is usually enough. It is lightning-fast, mathematically transparent, and requires no epochs or hyperparameter tuning. Deep Learning (the Multi-Layer ANN) is overkill for this specific dataset but serves as the perfect beginner bridge. Deep Learning shines when you have thousands of features (like pixels in an image or words in a document) and millions of rows, where simple lines cannot capture the hidden complexity.

Why Hidden Layers Matter

Hidden layers act as feature extractors. Without them, your model can only draw a straight line. With hidden layers and non-linear activation functions like ReLU, your model gains the ability to learn complex,

bending shapes in the data, uncovering deep interactions between variables that a human analyst might miss.

When to Use What?

Use Linear Regression (ML) when:

- Your dataset is small to medium-sized.
- You only have a handful of numerical features.
- You need high interpretability (you need to explain the math to a non-technical boss).
- You need instant training time.

Use Multi-Layer ANN (DL) when:

- Your dataset is massive (hundreds of thousands or millions of rows).
- You have incredibly complex, non-linear relationships in the data.
- You are dealing with unstructured data (images, text, audio) which is converted to numerical tensors.
- Raw accuracy is prioritized over interpretability.